FAMP
Executable Functional Memory

Executed pilot evidence and manuscript claim boundary

Protocol **famp-executable-memory-1.1.1**. The objective remains diagnosis-guided reusable functional learning with lower complete resource cost—not another gradient-projection or fault-localization framework.

What was verified	Observed execution
Sequential learners	120 runs; three independent seeds; arithmetic, nonlinear, and task-aware Digits
Notebook	20 code cells completed without errors; 32 startup controls
Round-trip verification	120 checkpoint reconstructions; 126 post-run assertions; maximum accuracy discrepancy 1.11e-16
Repeated execution	All 3,130 tensor payloads across 120 checkpoints match bitwise; all non-timing summaries match
Hardware	CPU execution. No CIFAR-100, multi-device throughput, peak-host-memory, or energy result is claimed

1. What is now implemented end to end?

The mutable fallback learns a task with residual replay. A candidate can reuse an existing frozen raw-input feature unit, compose two units, or allocate a new compact learned unit. One proposal is selected before independent acceptance. A complete-byte transaction stores unit references, a task head and bounded witnesses only after the risk, response, and capacity checks. Ordinary episodes for covered tasks are removed. Rejection retains episodic fallback.

The new retained object is executable: the same physical unit parameters can serve several task heads. The system no longer retains a full parameter-gradient basis or a permanent duplicate teacher. This verifies the intended storage mechanism, not that it beats all alternatives.

2. Main pilot outcomes

Family	Cap KiB	Packed ER %	Independent %	FAMP %	FAMP / ER time
arithmetic	24	91.60	92.45	92.56	2.00x
arithmetic	48	93.55	94.92	94.97	2.10x
nonlinear	24	89.76	90.68	90.49	2.05x
nonlinear	48	92.19	92.83	92.83	1.94x
digits	48	97.70	97.70	97.70	1.78x

Means are over three seeds. Within-cap comparisons use the same common fallback and labeled partitions. Published-state occupancy is close to the cap in most runs. These means are not validation-selected minimum-byte operating points. Digits is five task-aware binary problems, not the earlier ten-class class-incremental setting.

3. Is the benefit caused by sharing?

At the larger arithmetic cap, FAMP minus packed replay is **+1.42 [+0.04, +2.80] percentage points**. Against the otherwise identical independent-unit control, the difference is **+0.05 [-0.66, +0.76] points**. The former is a positive exploratory comparison; the latter does not establish a benefit specifically caused by sharing.

Family	KiB	FAMP − packed ER, pp [95% t CI]	FAMP − independent, pp [95% t CI]
arithmetic	24	+0.95 [-0.63, +2.53]	+0.11 [-0.20, +0.42]
arithmetic	48	+1.42 [+0.04, +2.80]	+0.05 [-0.66, +0.76]
nonlinear	24	+0.74 [-0.12, +1.59]	-0.18 [-1.40, +1.03]
nonlinear	48	+0.65 [+0.31, +0.98]	+0.00 [+0.00, +0.00]
digits	48	+0.00 [+0.00, +0.00]	+0.00 [+0.00, +0.00]

These are paired descriptive t intervals, not multiplicity-adjusted confirmation. Independent units use the same new-unit builder, risk check and witness policy, but cannot share old units. If that condition performs similarly, freezing compact task-specific solutions is a sufficient alternative explanation.

Across the 15 FAMP runs, the controller records **3 reuse/composition admissions**. Sharing therefore occurred, but it is sparse. The unit-ablation file records every dependent task, including cases in which deleting a contribution helps a particular input. It does not assign a semantic primitive name to a unit.

4. Why the current run is not a lower-footprint result

The experiment fills residual episodic storage up to the specified cap. A route can reduce its own episode requirement while the freed bytes are reassigned to unresolved tasks. That is a valid memory-allocation mechanism, but similar total occupancy is not a demonstrated reduction in the minimum memory needed at a target accuracy. A separate validation-selected frontier is required.

FAMP also trains its fallback and a new-unit candidate before selection, even when an old unit eventually wins. Search, head fitting, acceptance, witness checks, packing and ledger work are timed. In this CPU pilot, it is slower than packed replay. Fewer stored examples or fewer newly allocated units cannot turn this extra work into a compute saving by definition.

5. Why every gated Digits task falls back

The Digits acceptance split is class-stratified. The code uses a separate harmful-disagreement bound per class and a union bound, rather than an invalid pooled IID certificate. At the default five-task horizon, 5% family failure level, and 8% excess-harm tolerance, each class needs at least **64 acceptance examples even when zero harmful events are observed**. The small available class partitions do not satisfy that requirement.

All 15 FAMP Digits proposals therefore fail the harm-bound criterion and stay on replay. This is a calibration-data limitation of the selected confidence contract—not evidence that there is no reusable image computation. Ungated reuse does commit routes, but its average Digits accuracy is much lower. Removing the gate to produce more sharing would not validate the algorithm.

6. A measured shared-unit example, without cherry-picking

The first sharing event in the fixed run order is arithmetic seed 839, cap 24576 bytes, task 5 (zero-based). It composes physical unit IDs [0, 1]. The proposal was selected before its acceptance examples were examined.

Acceptance quantity	Measured value
Candidate accuracy	95.31%
Teacher accuracy	91.41%
Harmful disagreements	7 of 256
One-sided harmful-event upper bound	6.498% (threshold 8%)
Response RMS	0.195614 (threshold 0.20)
Per-task failure level	0.00555556

The worked-example artifact then inspects the first test input by index. Its true class is 0. The stored route produces logits approximately [−0.0520, −0.0244], hence the wrong class 1. Removing unit 0 changes the logits to [0.0769, −0.0269], which predicts class 0. This deliberately unselected example prevents interpreting an illustrative intervention as a guaranteed successful abstraction test.

Physical unit 0 serves tasks 0 and 5. Its weights are stored once. The task-5 head is a separately learned interface and is charged. Across a task distribution, the effect of removing a unit is evaluated in the exported ablation table; this single example is only an operational walkthrough.

Retained object	Bytes
Fallback weights	13,256
Unique units	2,240
Task heads	280
Witness tensors	312
Canonical metadata	998
Residual episodes	7,476
Total	24,562

This ledger is the final published state for that actual run, not an estimated comparison between a feature vector and a replay image. Serialized file overhead and transient candidate/optimizer workspace are different quantities. The same complete accounting applies to replay and independent-unit baselines.

7. What the paper is now allowed to say

Statement	Evidence status
The system stores and executes shared learned units	Implemented; unit IDs, hashes, heads and client-task ablations are exported
A hard published-state cap is enforced	All measured boundaries and reconstructed ledgers satisfy the cap
Admission has a conditional teacher-relative bound	Derived under independent/stratified sampling assumptions; not a semantic abstraction theorem
Sharing improves beyond independent compact units	Not established by this pilot
Lower total resource cost at competitive accuracy	Not established across the required benchmarks and hardware
Paper is ready for unchanged upload	No: empty confirmatory tables, public baselines, official-style replacement and human review remain

8. A fixed paper—not another research pivot

The manuscript fixes one algorithmic question: **when can validated shared executable memory replace episodic storage at a better accuracy–resource trade-off?** Diagnosis is an allocation decision, counterfactual consistency is an admission check, functional units are the retained objects, and replay is the fallback. The RQs separate those roles.

RQ	Decisive evidence	Where to fill
RQ1	Competitive accuracy at a smaller measured full cap or lower end-to-end cost	Main Table 1
RQ2	Gain over otherwise identical non-sharing units and clean composition ablation	Main Table 2
RQ3	Reliable independent harmful-event and response audits, not merely greater acceptance	Main Table 3
RQ4	All objects, temporary workspace, failed searches, I/O and serving latency	Main Table 4; Appendix Table A3
RQ5	Honest refusal under weak teachers, small calibration, shifts and unrelated/long streams	Main Table 5

The paper tables stay empty. The registry marks which fields are implemented in the pilot and which require additional observer runners, competitive public baselines, or hardware measurements. Public-method names cannot be assigned to the simple internal codecs. The final claim must be driven by those contrasts, not by the existence of a compiling paper or a successful unit transaction.

Reproducibility receipts and source files

Start with **FAMP_Executable_Functional_Memory.ipynb**. Its final sections reconstruct checkpoints, show the measured example, regenerate the report and export an archive. **all_runs.csv** provides summary measurements; **events.json** records proposal outcomes; per-run **ledger.csv** files account for each stage; **reuse_ablation.csv** records dependency tests.

The exact source hash, environment, startup assertions, round-trip checks, repeated-execution comparison, primary-source bibliography audit and document-validation receipt accompany the package. The second execution is a reproducibility test and does not double the number of independent seeds.